Mapeo de Datos Complejos en el Modelo Multidimensional

1. El espacio vectorial de las imágenes

Imagina una foto de 256 × 256 a color. Si cada píxel de cada canal es una variable, esa foto se convierte en un vector de 196,608 dimensiones. Casi 200 mil variables para una sola imagen. Y encima los píxeles vecinos se parecen muchísimo entre sí, así que hay mucha información repetida. Por eso las redes convolucionales no van píxel por píxel: usan filtros que comparten pesos y van achicando la dimensión capa por capa.

2. El espacio vectorial del texto (TF-IDF y Word Embeddings)

Con TF-IDF, cada documento es un vector con una dimensión por palabra del vocabulario. La fórmula tf(t,d) × log(N / df(t)) le da más peso a las palabras que se repiten en un texto pero casi no aparecen en los demás. El problema es que salen vectores gigantes, llenos de ceros, que no entienden nada de significado: para TF-IDF, “feliz” y “contento” son tan distintos como “feliz” y “mesa”.

Los Word Embeddings (Word2Vec, GloVe, BERT) arreglan eso. Una red neuronal aprende vectores más compactos, de 100 a 768 dimensiones, según el contexto de cada palabra. Así, las palabras parecidas quedan juntas y hasta puedes hacer cuentas como rey − hombre + mujer ≈ reina. Los Transformers aplican la misma idea a oraciones completas. Para compararlos se usa la similitud coseno, porque importa hacia dónde apunta el vector, no su tamaño.

3. La maldición de la dimensionalidad

Tener tantas dimensiones trae tres problemas. Primero, el espacio se queda vacío: para agarrar solo el 10% de los datos con 10 dimensiones, necesitas cubrir el 79% del rango de cada variable, así que lo “local” ya no es nada local. Segundo, las distancias se emparejan: el vecino más cercano y el más lejano quedan casi a la misma distancia, y todos los usuarios de la matriz X parecen igual de cerca. Tercero, el ruido gana: las dimensiones que no aportan nada meten ruido, el modelo se sobreajusta y falla con datos nuevos.

Conclusión

Sí, las imágenes y el texto se pueden convertir en vectores, pero terminan en espacios enormes donde los algoritmos basados en distancia dejan de funcionar. Por eso, antes de modelar, conviene reducir dimensiones con PCA, convoluciones o embeddings densos.